

Software Ontwerp

Naam: Lukas Hoogendam
Studentnummer: s1142688
Vak: First full stack
Datum: 11-03-2025
Versie: 1.1

Inhoudsopgave

Inleiding	4
Applicatie ontwerp.....	5
Database ontwerp.....	6
Dataopslag.....	6
Entiteiten	6
Normalisatie	6
Relaties	6
ERD diagram	7
Frontend architectuur	8
Klassen diagram	9
Admin	10
Auth	11
Product	12
Order.....	13
File-structuur	14
Component tree	15
Main layout	16
Admin layout.....	16
Backend architectuur.....	17
Klassen diagram	18
User	19
Product	20
Category.....	21
Brand	22
Cart.....	23
JWT/Security	24
Sequentie diagrammen	25
Login flow	25
Registratie flow	25
Cart merge flow.....	26
Producten ophalen flow.....	26
Order flow.....	27
Item toevoegen aan cart flow	27
End-points en benodigde waardes	28
Authenticatie	28
Producten.....	28
Categorieën	29
Merken	29
Winkelwagen	30
Bestellingen	30
Gebruikers.....	31
Response DTO's	32
Security	34
Frontend security OWASP	34

Backend security	35
Authenticatie	35
JWTFilter	35
Autorisatie	35
Wachtwoordbeveiliging	35
Inputvalidatie	35
CORS	36
CSRF	36
Stateless sessies	36
Gastwinkelwagen	36

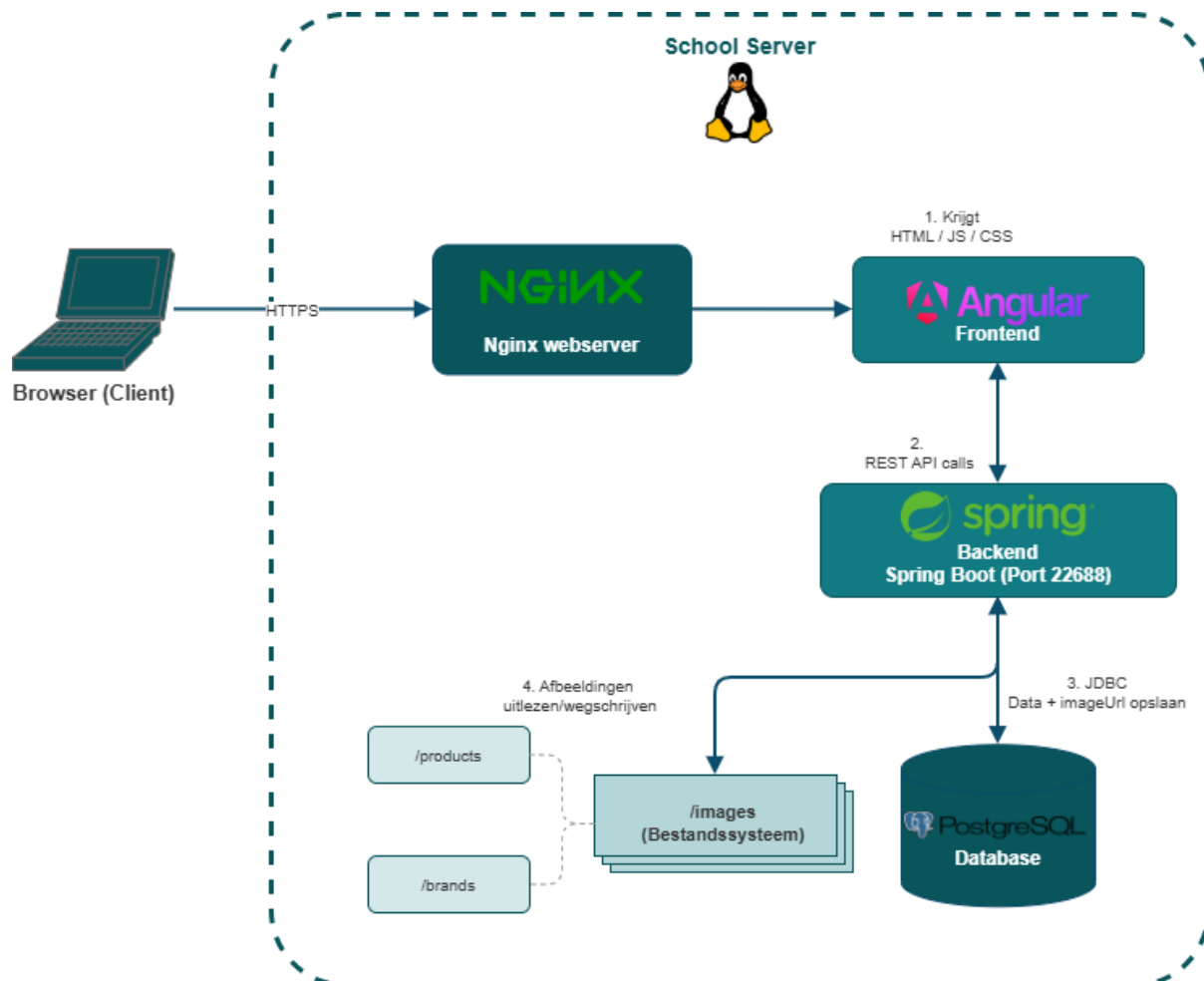
Inleiding

Dit ontwerp is gemaakt voor een webshop applicatie voor de module InfirSFS. De applicatie bestaat uit een Angular frontend en een Spring Boot backend waarmee gebruikers producten kunnen bekijken, toevoegen aan een winkelwagen en bestellingen kunnen plaatsen. Daarnaast is er een adminpaneel waarmee beheerders producten, categorieën, merken en gebruikers kunnen beheren. In dit document worden de technische keuzes, de architectuur, de klassendiagrammen, sequentiediagrammen en de security van de applicatie beschreven.



Applicatie ontwerp

De applicatie draait op een schoolserver. Nginx serveert de Angular frontend als statische bestanden naar de browser. Vanuit de frontend worden REST API calls gedaan naar de Spring Boot backend, die data opslaat in een PostgreSQL database en afbeeldingen leest en schrijft in de /images map op het bestandssysteem.



Database ontwerp

Dataopslag

De data wordt opgeslagen in een PostgreSQL database.

Entiteiten

De USER tabel slaat alle gebruikers op. Via het veld role wordt bijgehouden of iemand een klant of admin is. De PRODUCT tabel bevat alle producten met een prijs, beschrijving, voorraad en afbeelding. Elk product is gekoppeld aan een CATEGORY en een BRAND via foreign keys. De CART tabel stelt een winkelwagen voor, die gekoppeld is aan een gebruiker of aan een sessietoken voor gastgebruikers. De losse producten in de winkelwagen staan in CART_ITEM. Hetzelfde patroon geldt voor bestellingen: de ORDER tabel slaat de bestelling op en ORDER_ITEM bevat de losse producten per bestelling.

Normalisatie

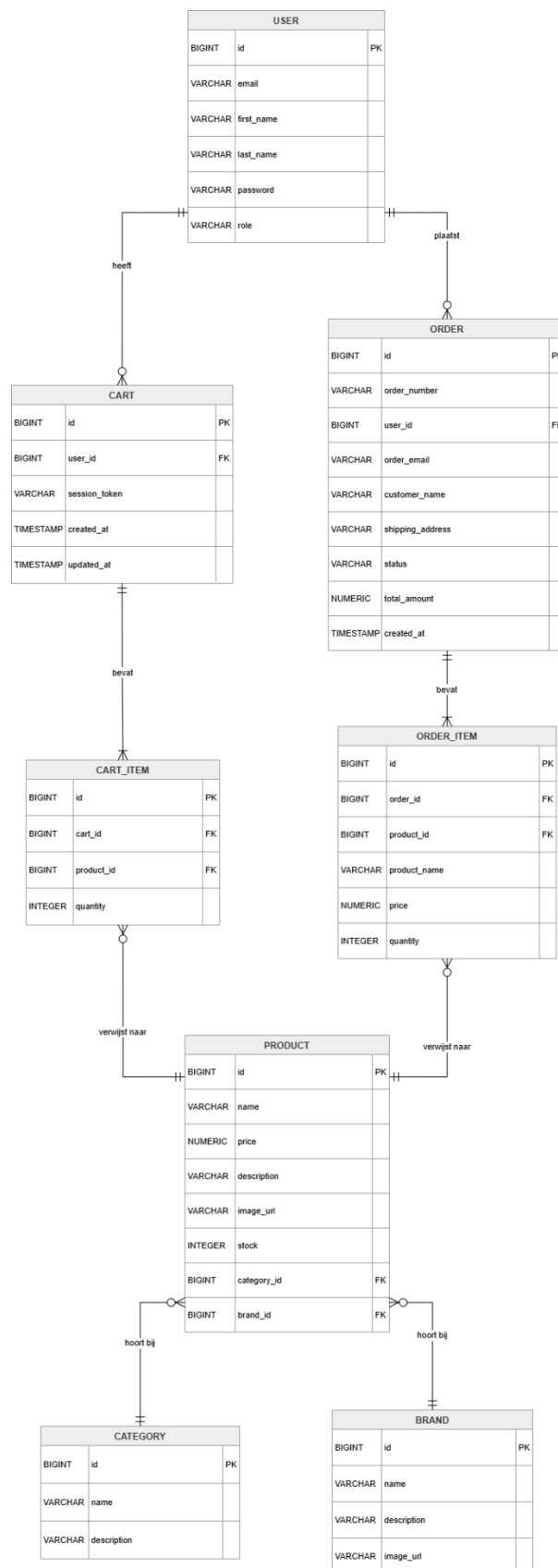
De database is genormaliseerd tot de derde normaalvorm (3NF). Dat betekent dat alle gegevens maar op één plek staan en er geen overbodige herhalingen zijn. Zo worden merknaam en categorienaam niet herhaald in de producttabel, maar staan ze in aparte tabellen. Hetzelfde geldt voor gebruikersgegevens: die staan alleen in de USER tabel en worden nergens gekopieerd.

Relaties

De relaties in de database zijn belangrijk voor de consistentie van de data. De koppeling tussen CART_ITEM en PRODUCT zorgt ervoor dat een product maar één keer gedefinieerd hoeft te worden, maar in meerdere winkelwagens en bestellingen voor kan komen. De koppeling tussen ORDER en USER maakt het mogelijk om de bestelgeschiedenis van een gebruiker op te halen. De koppeling via een sessietoken in CART zorgt ervoor dat ook gastgebruikers een winkelwagen kunnen hebben zonder dat ze een account nodig hebben.

ERD diagram

Hieronder volgt een ERD diagram die de entiteiten en relaties weergeeft.



Frontend architectuur

De frontend is gebouwd met Angular in TypeScript en volgt een gelaagde architectuur. Dit patroon lijkt op MVC: services vormen de data laag, componenten de presentatielaag en de router bepaalt welke pagina wordt getoond. Een component haalt nooit zelf data op, maar vraagt dit altijd via een service.

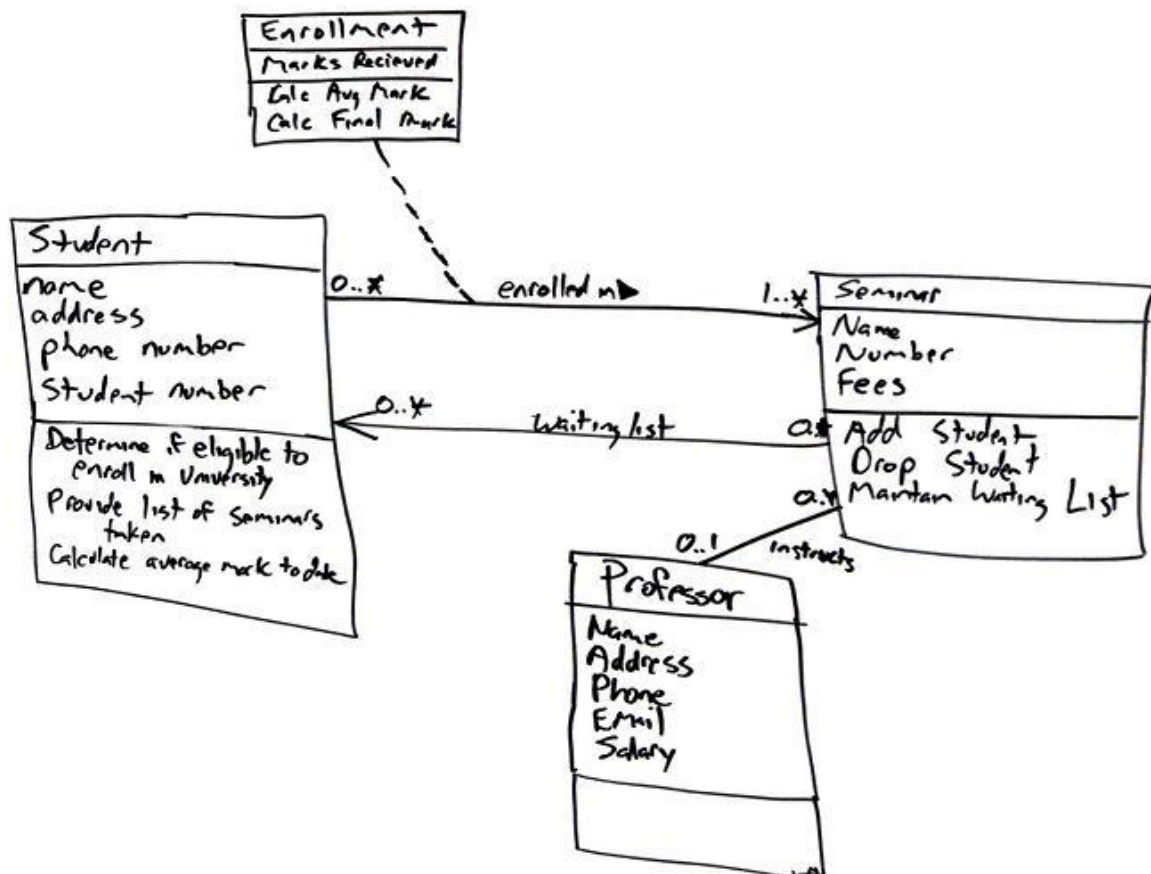
Alle communicatie met de backend gaat via de ApiService. Andere services zoals AuthService en CartService bouwen hier op voort. Voor state management worden Angular Signals gebruikt, zodat componenten automatisch bijwerken als data verandert.

Pagina's worden pas ingeladen wanneer je er naartoe navigeert via lazy loading. Beveiligde pagina's zijn afgeschermd met guards: authGuard voor ingelogde gebruikers en adminGuard voor admins.

Klassen diagram

Een klassendiagram is een soort tekening die laat zien hoe de code is opgebouwd. Je ziet welke klassen en interfaces er zijn, welke velden en methodes ze hebben, en hoe ze met elkaar verbonden zijn. Het helpt om snel te begrijpen hoe de applicatie in elkaar zit zonder alle code te lezen.

Hieronder worden de klassendiagrammen per domein beschreven



→
Directed association

→
Inheritance

◆
Composition

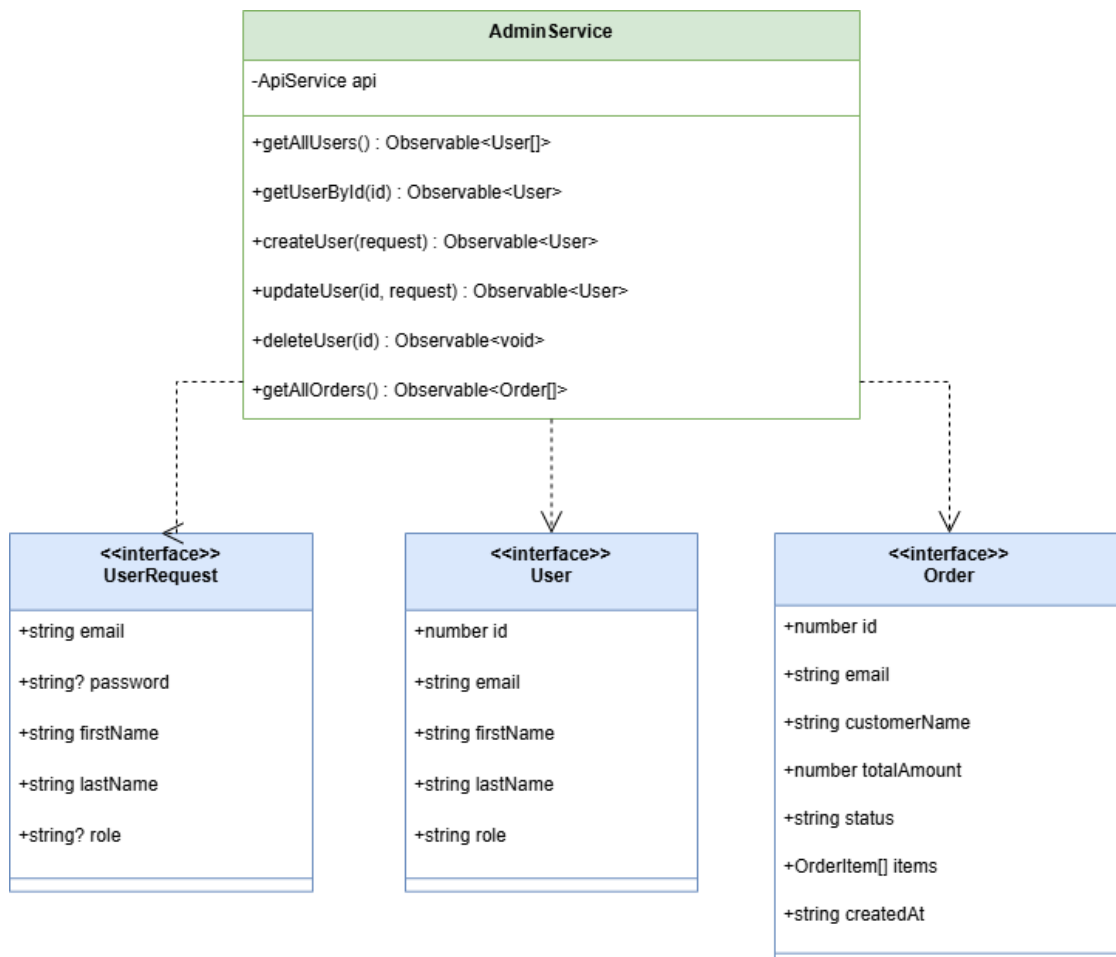
- - ->
Realization/Implementation

- - -◇
Aggregation

- - - - ->
Dependency

Admin

Het admin diagram laat zien hoe de adminpagina gebruikers en bestellingen beheert. De AdminService is de centrale service en heeft methodes om gebruikers op te halen, aan te maken, te bewerken en te verwijderen. Daarnaast kan hij alle bestellingen ophalen. De AdminService werkt met twee bestaande modellen: User en Order. Naast de bestaande User interface is er ook een UserRequest interface, die wordt gebruikt als je een nieuwe gebruiker aanmaakt of een bestaande aanpast.

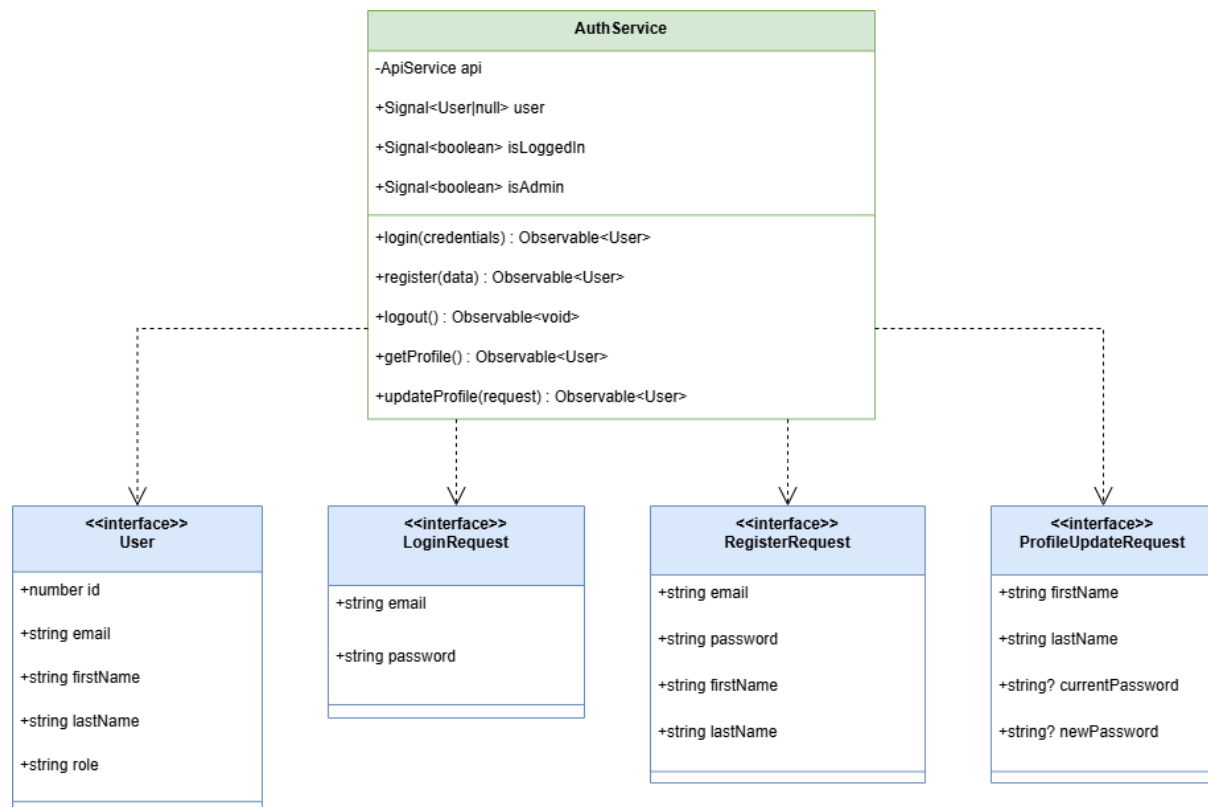


Auth

Het auth diagram beschrijft alles rondom inloggen en registreren.

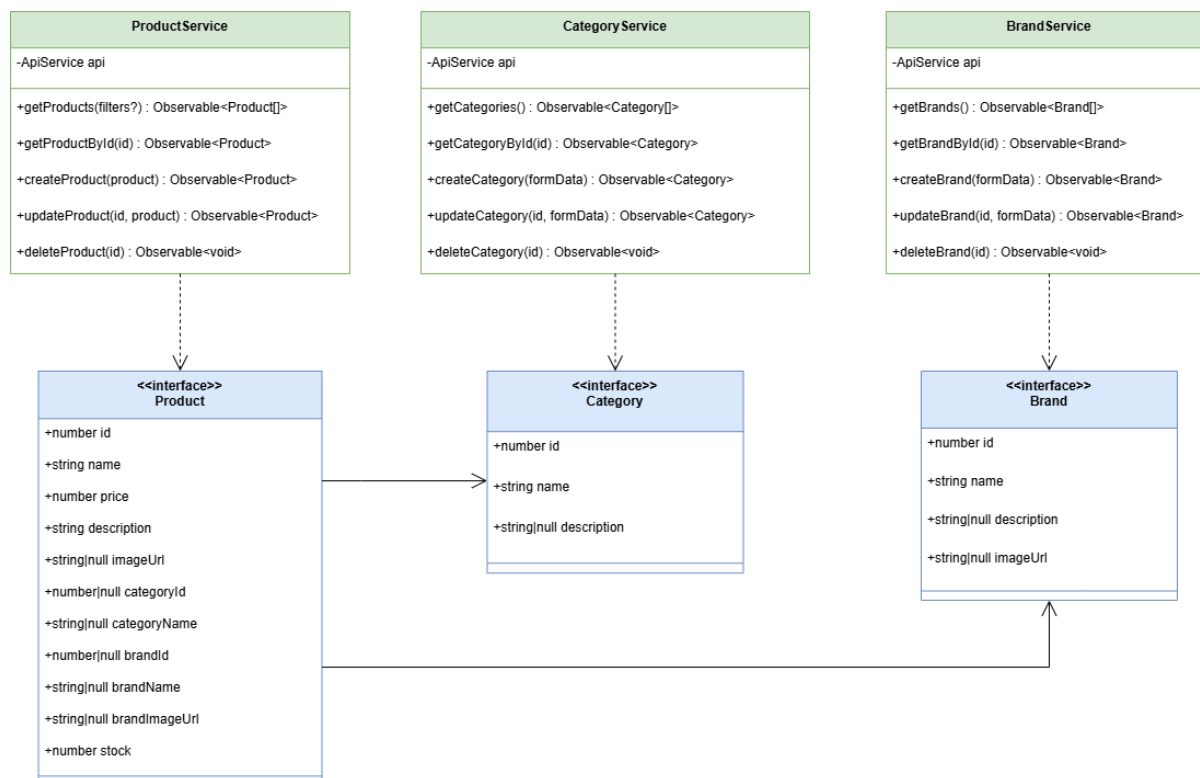
De AuthService beheert of een gebruiker is ingelogd en of hij een admin is, via Angular Signals. Hij heeft methodes om in te loggen, uit te loggen, te registreren en het profiel op te halen of bij te werken. De drie request-interfaces

(LoginRequest, RegisterRequest, ProfileUpdateRequest) geven aan welke data je meestuurt bij die acties. Het model User bevat de data die je terugkrijgt na het inloggen.



Product

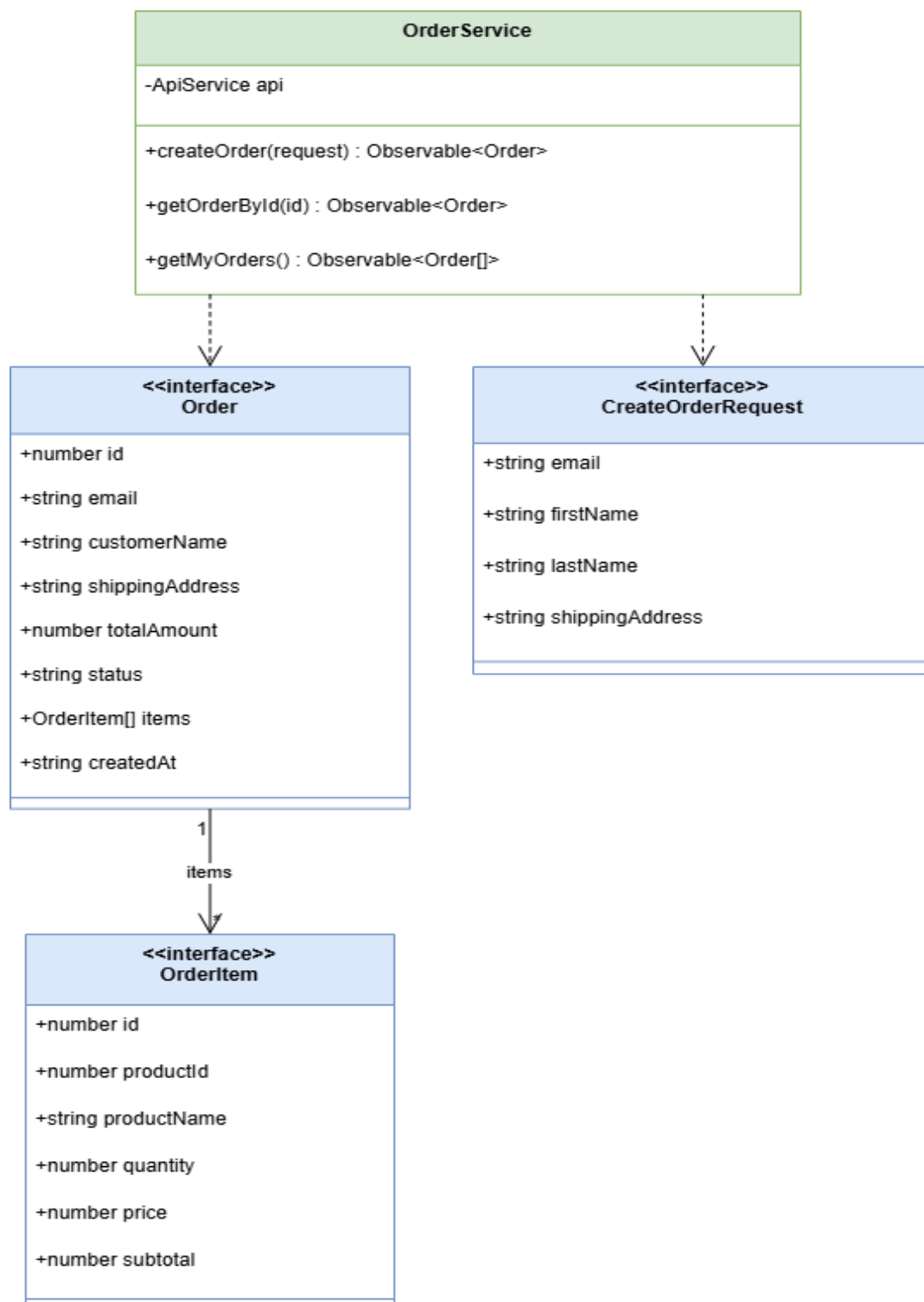
Het product diagram toont drie modellen en drie services. Product is het hoofdmodel en verwijst naar Category en Brand. De ProductService haalt producten op en beheert ze. BrandService en CategoryService doen hetzelfde voor merken en categorieën. Deze splitsing zorgt ervoor dat elke service verantwoordelijk is voor zijn eigen model, wat de code overzichtelijk houdt.



Order

Het order diagram laat zien hoe een bestelling is opgebouwd. Een Order bevat meerdere OrderItem objecten, elk voor één product in de bestelling.

De OrderService heeft methodes om een nieuwe bestelling aan te maken, een specifieke bestelling op te halen en alle eigen bestellingen te bekijken. CreateOrderRequest is de interface die je invult als je een bestelling plaatst, met gegevens zoals naam en afleveradres.



File-structuur

Voor de structuur van mijn Angular project heb ik inspiratie opgedaan uit het artikel van Atanasoae (2023) over het opzetten van een Angular project. Het idee is om de code op te delen in een paar hoofdmappen, zodat alles makkelijk te vinden is en het project overzichtelijk blijft naarmate het groeit. Mijn src/app/ map is opgedeeld in vier hoofdmappen: core, shared, features en layout.

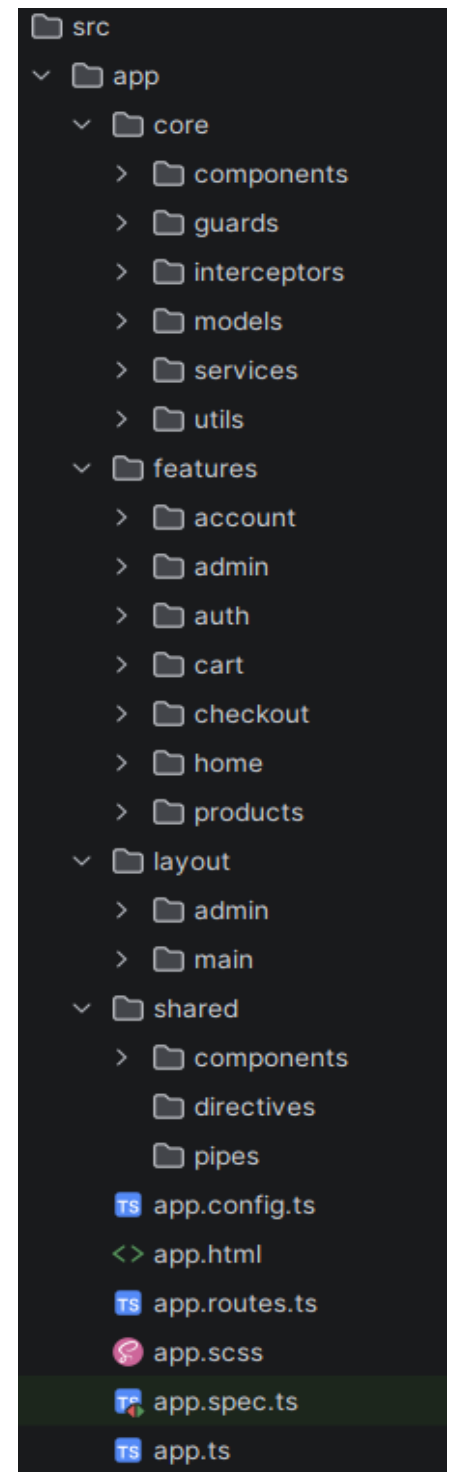
In de core map staat alles wat maar een keer bestaat in de hele app en overal gebruikt wordt. Hier zitten de services zoals AuthService, CartService en ProductService. Ook de models (zoals User, Product, Order en Cart), de guards (AuthGuard en AdminGuard), de interceptors en kleine hulpfuncties staan hier. Deze map is het fundament van de app.

De shared map bevat onderdelen die op meerdere plekken terugkomen, zoals herbruikbare componenten. Op dit moment staat hier bijvoorbeeld een zoekbalk component die in verschillende features wordt gebruikt.

De features map is waar het grootste deel van de app leeft. Elke feature heeft zijn eigen map, zoals home, products, cart, checkout, auth, account en admin. Binnen een feature staan de bijbehorende pagina's, en soms ook eigen componenten of services die alleen bij die feature horen. Zo heeft de admin feature bijvoorbeeld eigen services en pagina's, en de products feature eigen componenten. Door deze aanpak is elke feature op zichzelf staand en hoef je niet door het hele project te zoeken.

De layout map bevat de visuele omhulsels van de app. Er zijn twee layouts: een main layout met een header en footer voor de gewone gebruiker, en een admin layout met een sidebar voor het beheerderspaneel.

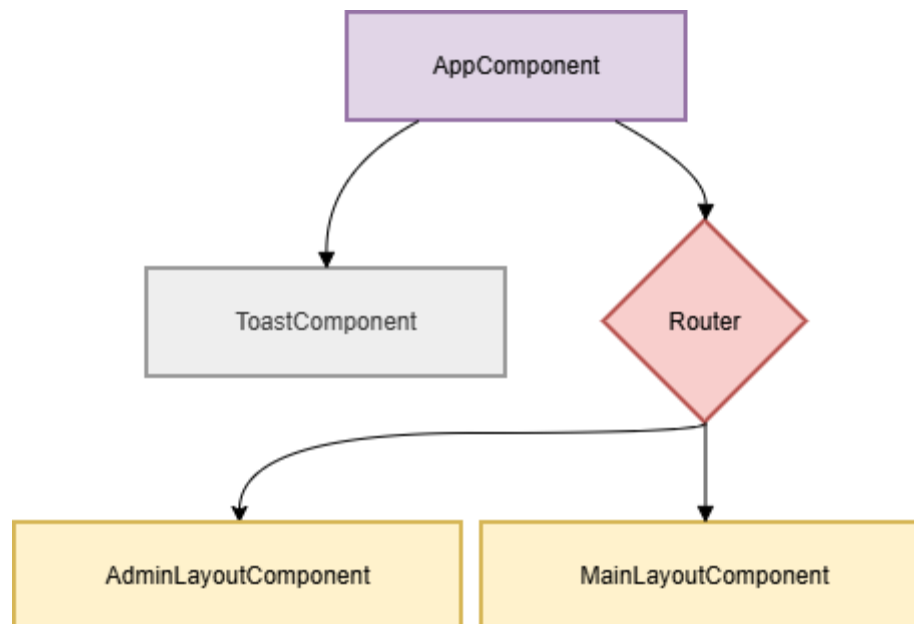
Door deze opzet te volgen is de code netjes gescheiden per verantwoordelijkheid en kan het project makkelijk groeien zonder dat het rommelig wordt.



Bron: Atanasoae, D. (2023). *Angular Project Structure Guide: Small, Medium, and Large Projects*. Medium. https://medium.com/@dragos.atanasoae_62577/angular-project-structure-guide-small-medium-and-large-projects-e17c361b2029

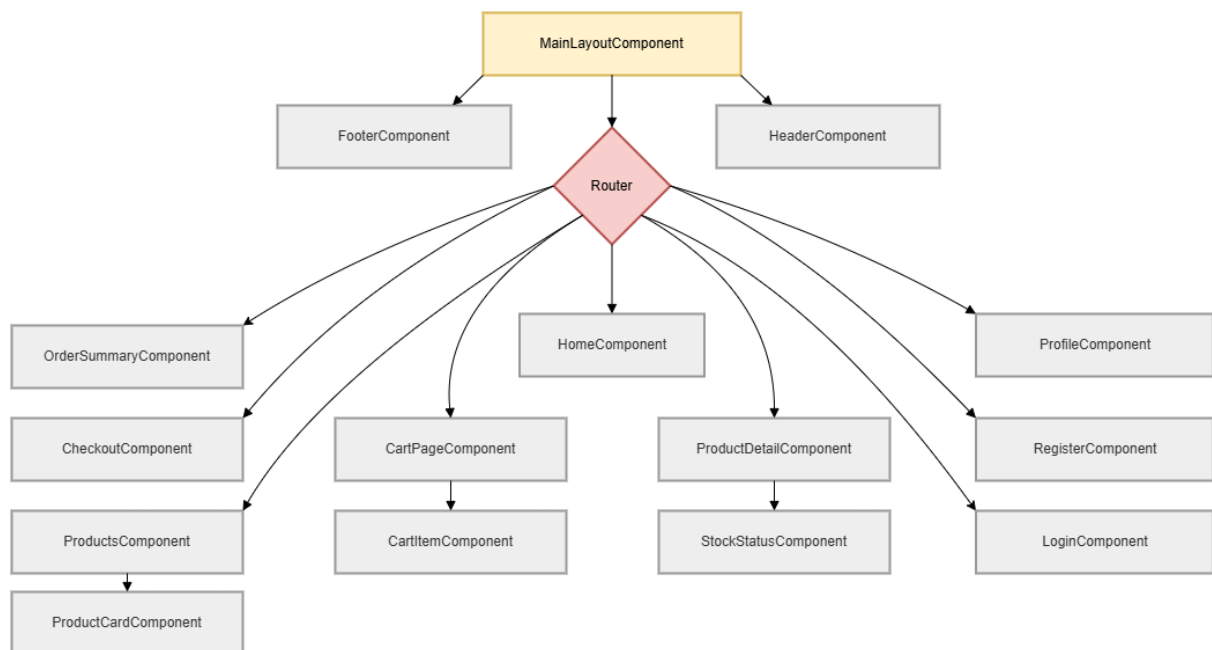
Component tree

De component tree heb ik in 3 diagrammen opgesplitst. Zoals in de file structuur beschreven staat, werk ik met layouts. De main layout en de admin layout.

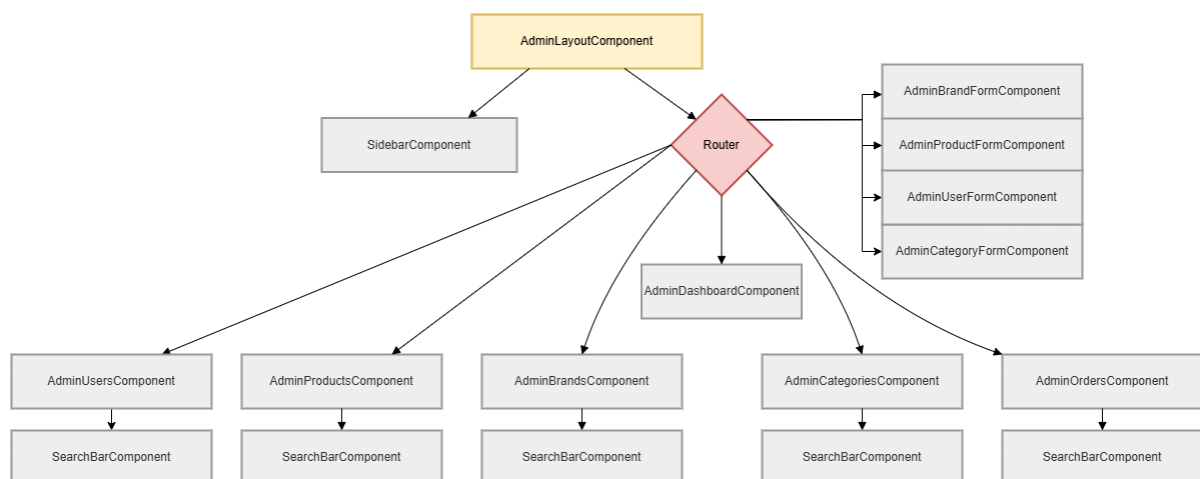


Toast component is een melding component om error of succes messages te tonen. De Router kan wisselen tussen de twee layouts. De Admin layout zal altijd met een admin guard beveiligd worden.

Main layout



Admin layout



Backend architectuur

De backend van de applicatie is gebouwd met Spring Boot in Java. De architectuur volgt een gelaagde structuur die je veel terugziet in professionele projecten. Elk request gaat door meerdere lagen heen: eerst komt het binnen bij een Controller, die het doorstuurt naar een Service voor de business logica, en uiteindelijk wordt de data opgehaald of opgeslagen via een Repository. Dit patroon wordt ook wel het MVC-patroon genoemd. Daarnaast worden Mappers gebruikt om database-entiteiten om te zetten naar DTO's (Data Transfer Objects), zodat er nooit meer data naar de frontend wordt gestuurd dan nodig is.

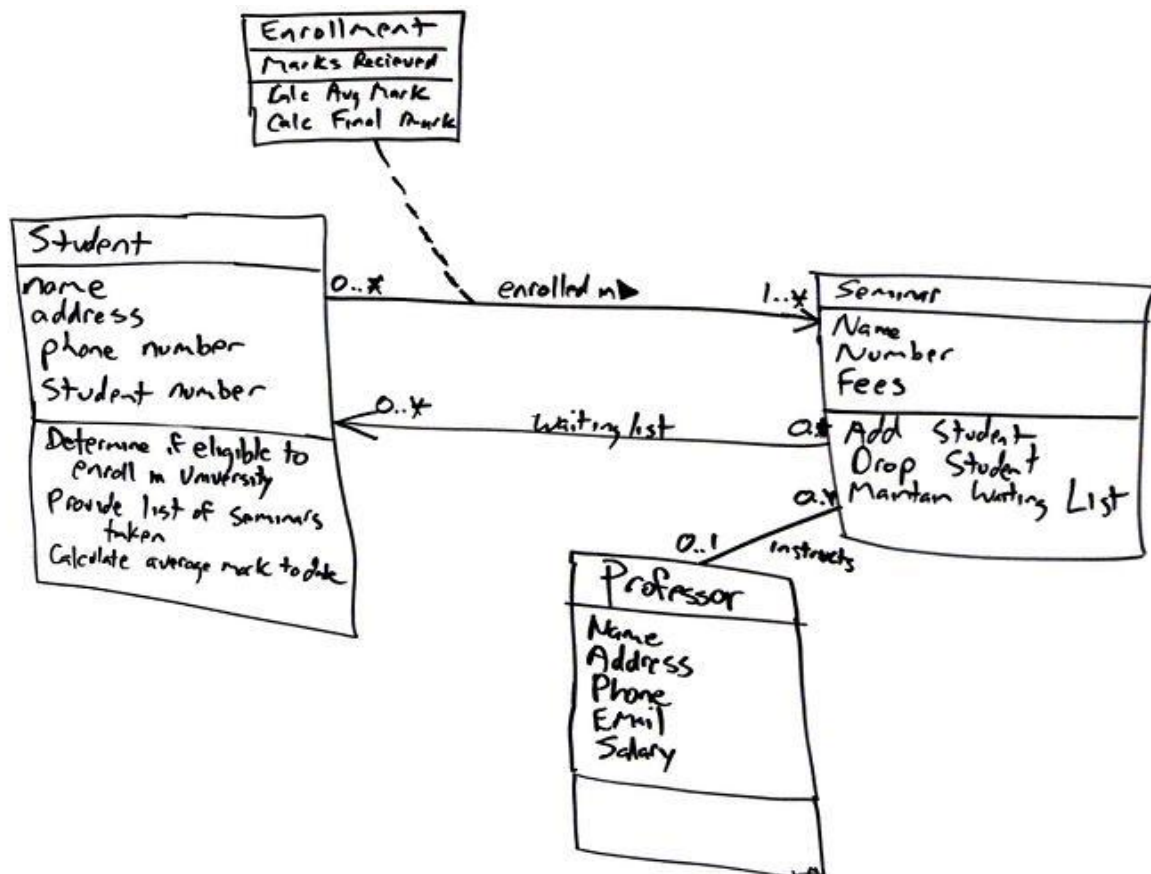
De backend is opgedeeld in meerdere domeinen: Product, Category, Brand, Cart, Order, User en Security. Elk domein heeft zijn eigen Controller, Service, Repository, Mapper en DTO's. Dit zorgt ervoor dat de code overzichtelijk blijft en dat elk onderdeel los van elkaar te onderhouden is.

Voor de beveiliging wordt Spring Security gebruikt in combinatie met JWT tokens. Gebruikers krijgen bij het inloggen een token in een HttpOnly cookie, en bij elk volgend request wordt die cookie automatisch meegestuurd en gecontroleerd.

Klassen diagram

Een klassendiagram is een soort tekening die laat zien hoe de code is opgebouwd. Je ziet welke klassen en interfaces er zijn, welke velden en methodes ze hebben, en hoe ze met elkaar verbonden zijn. Het helpt om snel te begrijpen hoe de applicatie in elkaar zit zonder alle code te lezen.

Hieronder worden de klassendiagrammen per domein beschreven



—————>
Directed association

—————>|
Inheritance

—————◆
Composition

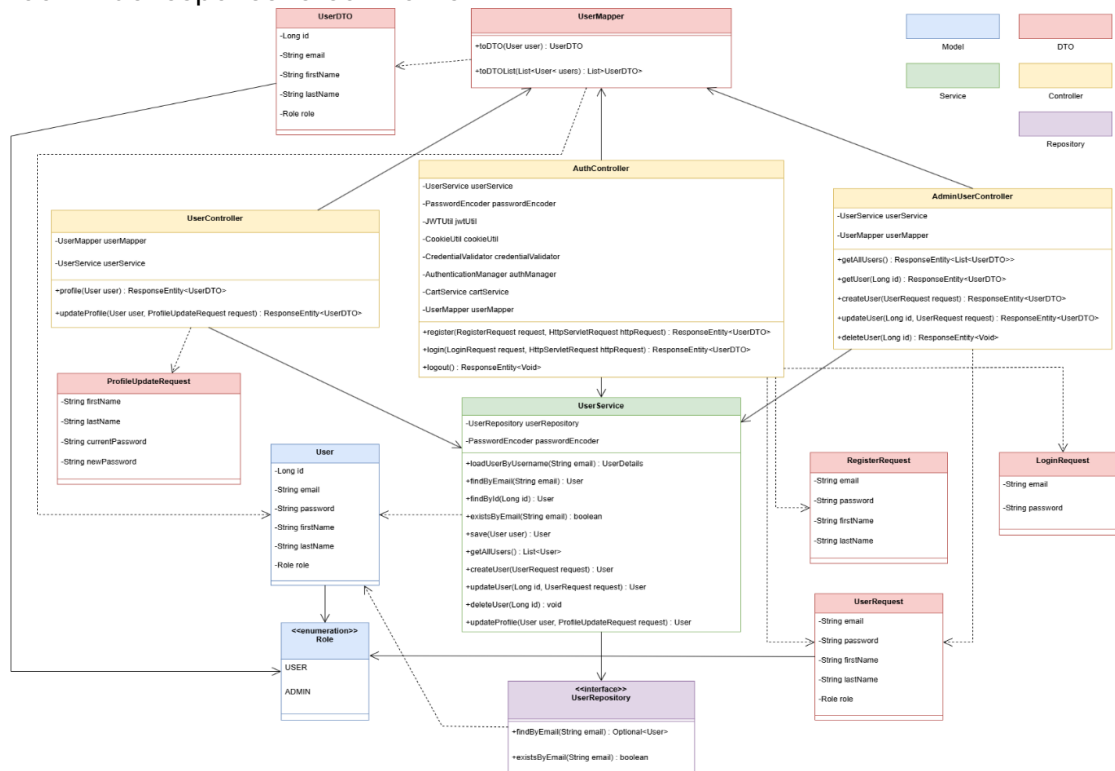
- - - - ->|
Realization/Implementation

—————◇
Aggregation

- - - - ->
Dependency

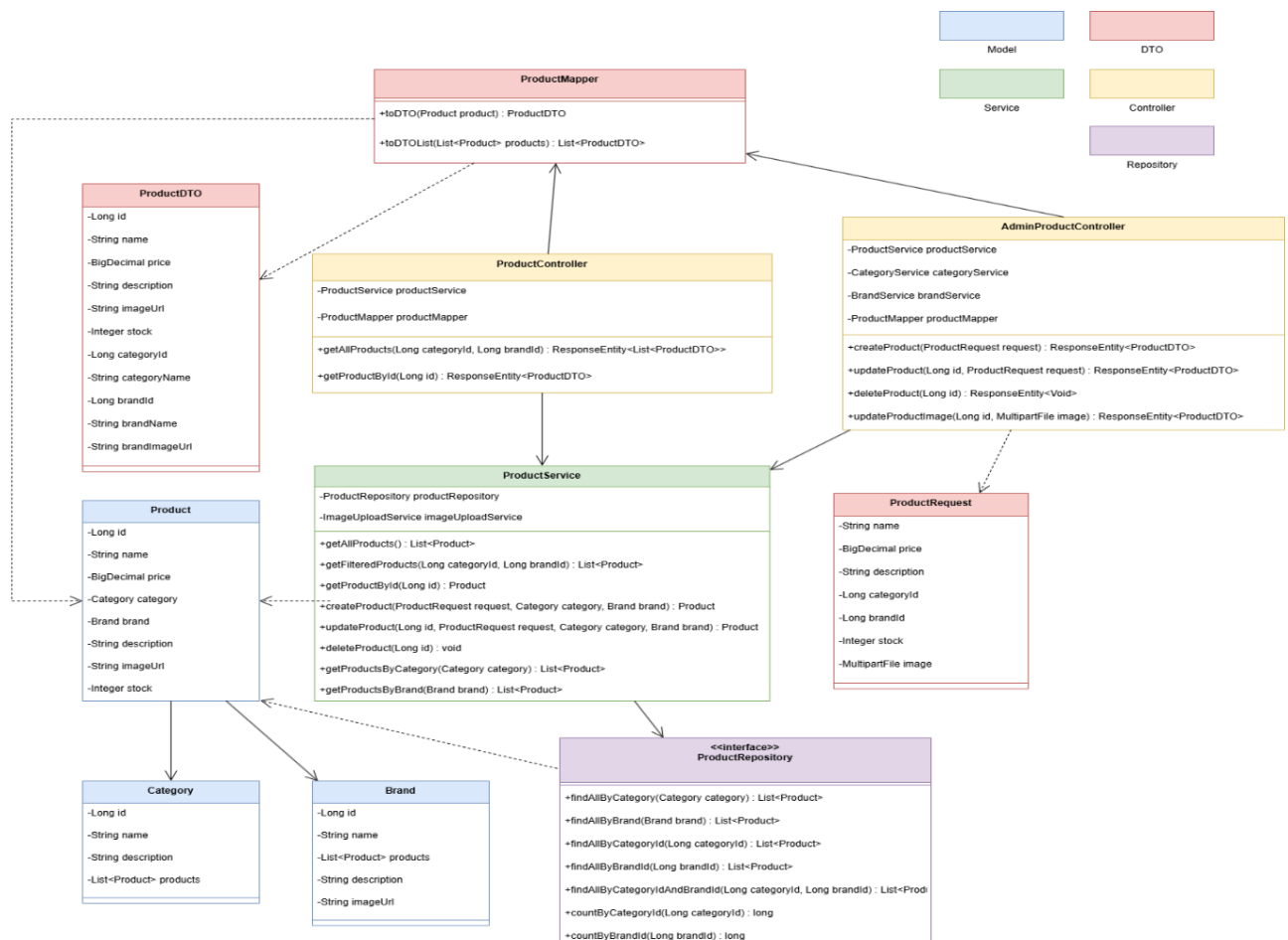
User

Het User-domein bevat de UserController waarmee een ingelogde gebruiker zijn eigen profiel kan ophalen en bijwerken. De AdminUserController geeft admins de mogelijkheid om alle gebruikers te bekijken, aan te maken, bij te werken en te verwijderen. De UserMapper zorgt ervoor dat gevoelige gegevens zoals het wachtwoord nooit in de response terechtkomen.



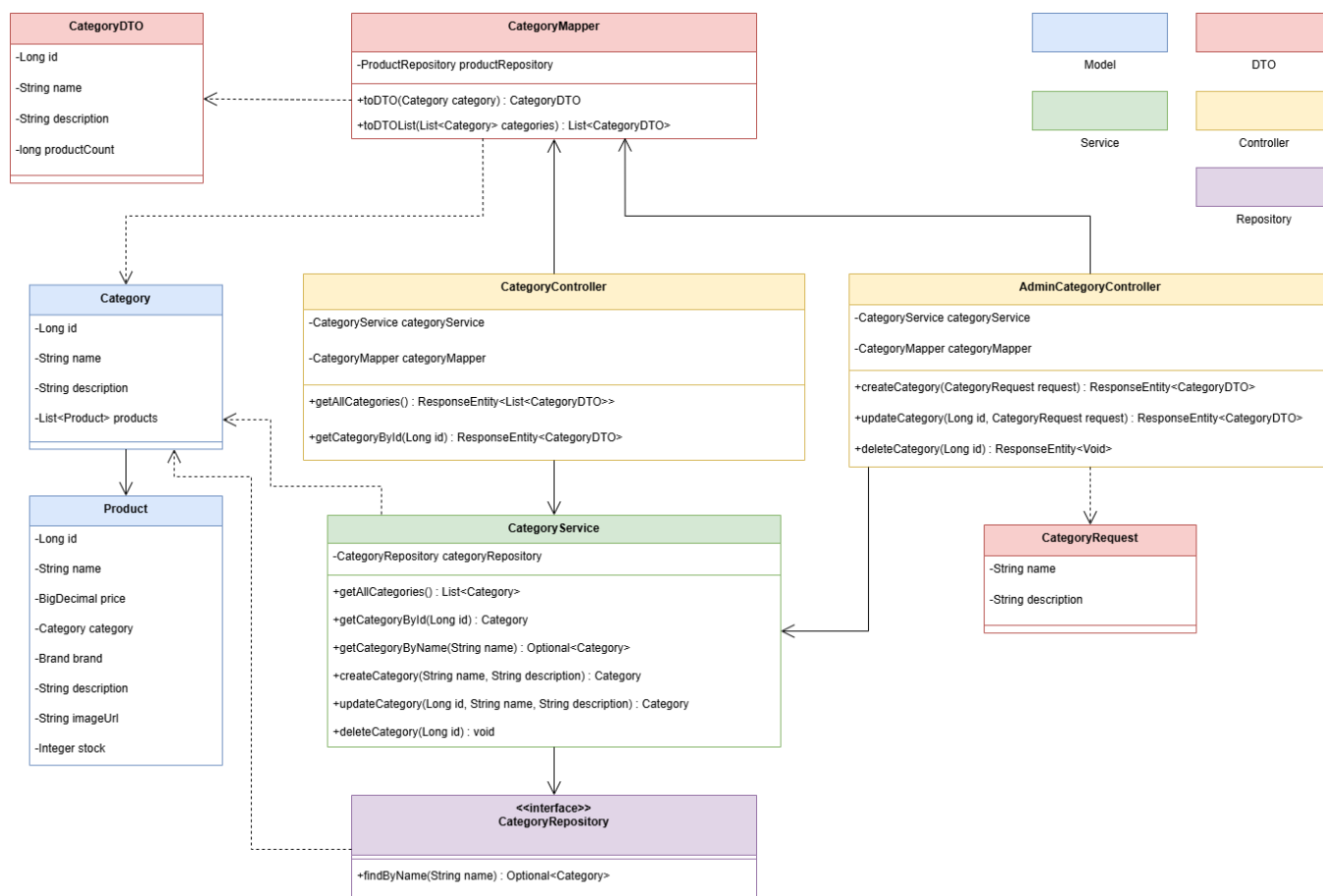
Product

Het klassendiagram van Product laat zien hoe een product wordt beheerd in de backend. De ProductController ontvangt requests en stuurt ze door naar de ProductService. De service gebruikt de ProductRepository om producten op te halen of op te slaan in de database. De ProductMapper zorgt ervoor dat het Product-model wordt omgezet naar een ProductDTO voordat het naar de frontend gaat. Er is ook een AdminProductController die alleen toegankelijk is voor admins, waarmee producten aangemaakt, bijgewerkt en verwijderd kunnen worden.



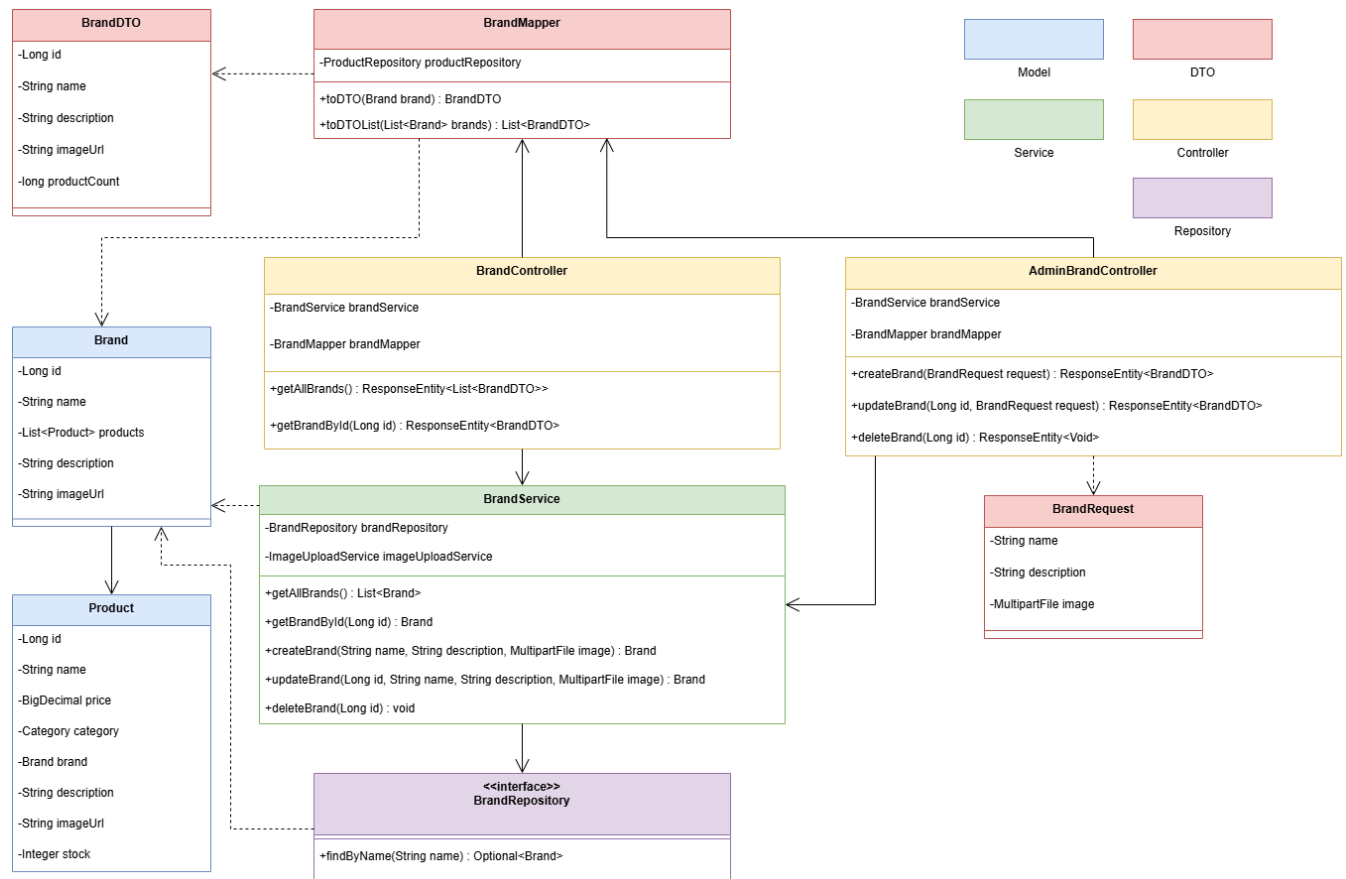
Category

Het klassendiagram van Category is vergelijkbaar met dat van Product maar dan simpeler. De CategoryController biedt endpoints om alle categorieën op te halen of een specifieke categorie op ID. De AdminCategoryController geeft admins de mogelijkheid om categorieën aan te maken, bij te werken en te verwijderen. De CategoryMapper zet het Category-model om naar een CategoryDTO.



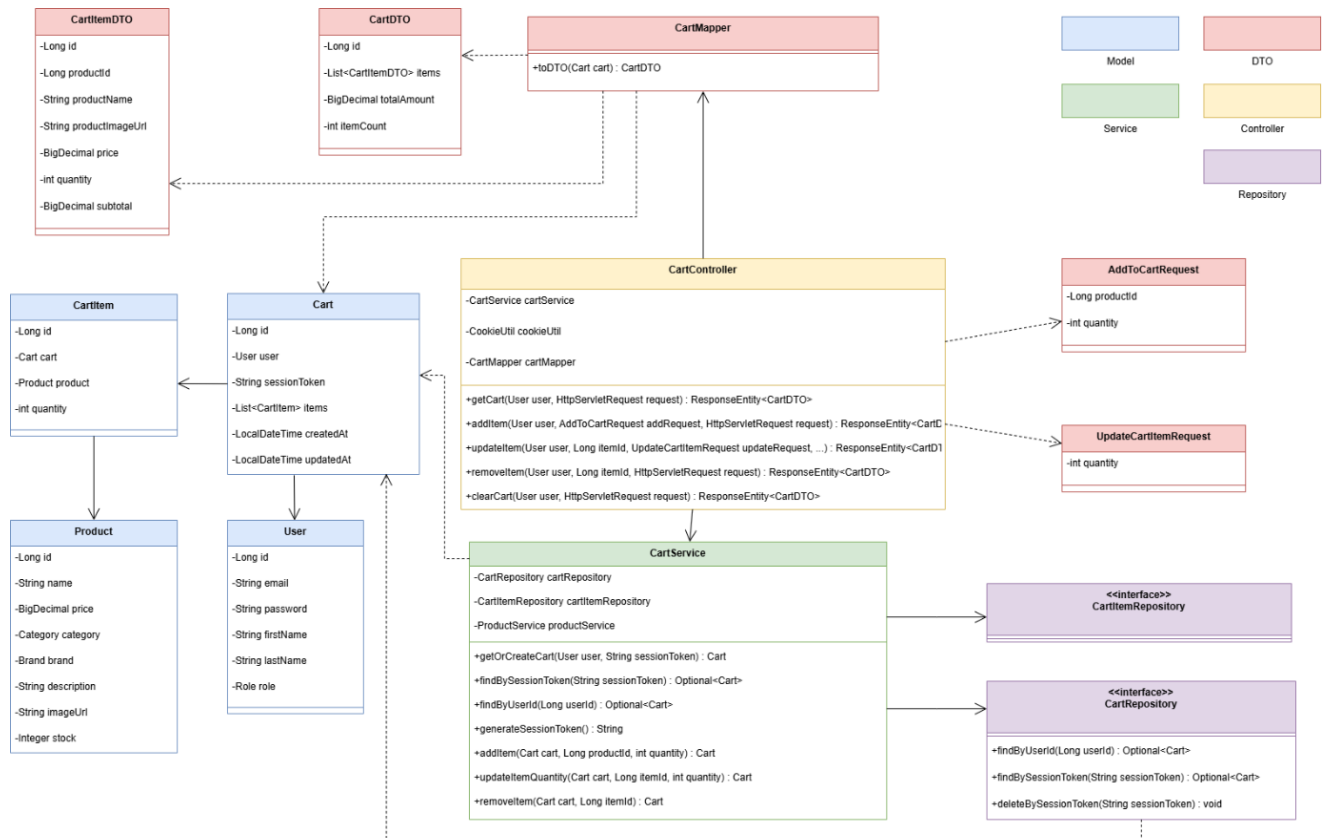
Brand

Het Brand-domein werkt op dezelfde manier als Category. De BrandController laat iedereen merken ophalen, en de AdminBrandController is alleen voor admins om merken te beheren. Omdat merken ook een afbeelding kunnen hebben, worden de admin-endpoints als multipart/form-data verstuurd. De BrandMapper converteert het Brand-model naar een BrandDTO.



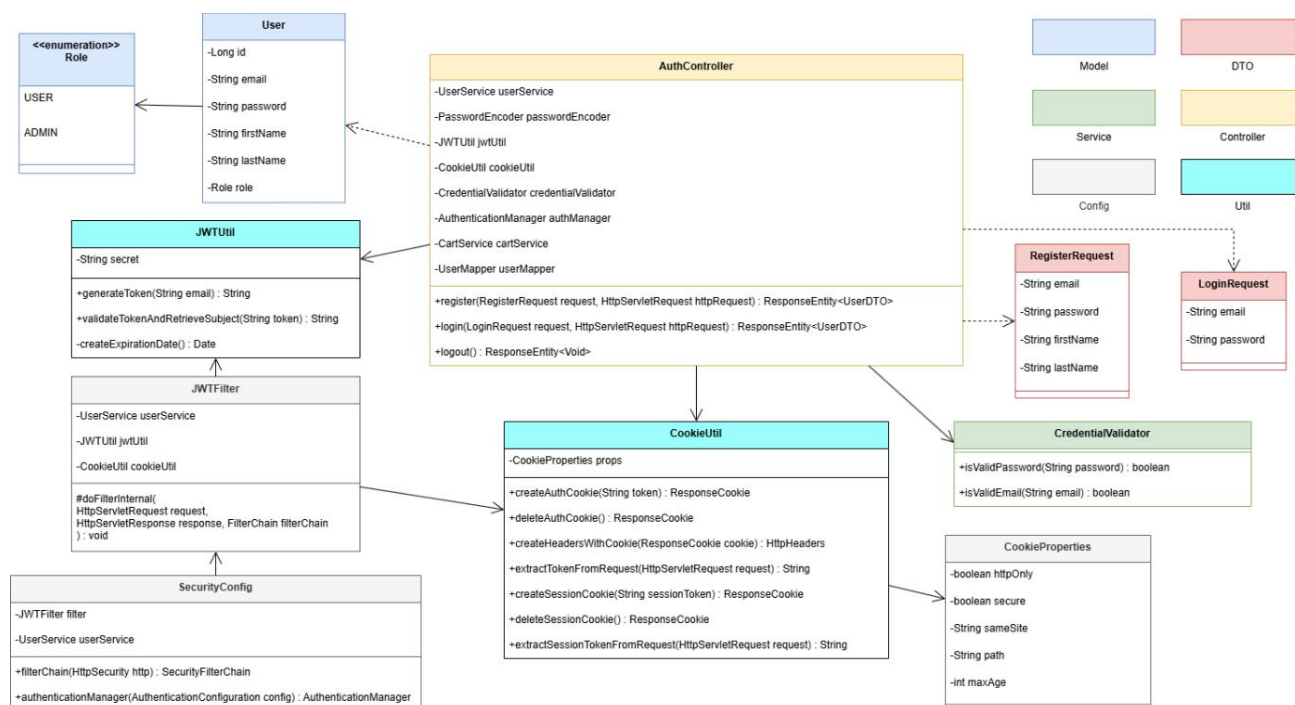
Cart

Het klassendiagram van Cart is iets complexer omdat de winkelwagen werkt voor zowel ingelogde gebruikers als gasten. De CartController handelt alle winkelwagen-acties af, zoals items toevoegen, bijwerken en verwijderen. De CartService bevat de logica voor het aanmaken van een cart, het controleren van voorraad en het samenvoegen van een anonieme cart met een gebruikerscart bij het inloggen. CartItems zijn de losse producten in de winkelwagen. De CartMapper zet alles om naar een CartDTO met CartItemDTO's.



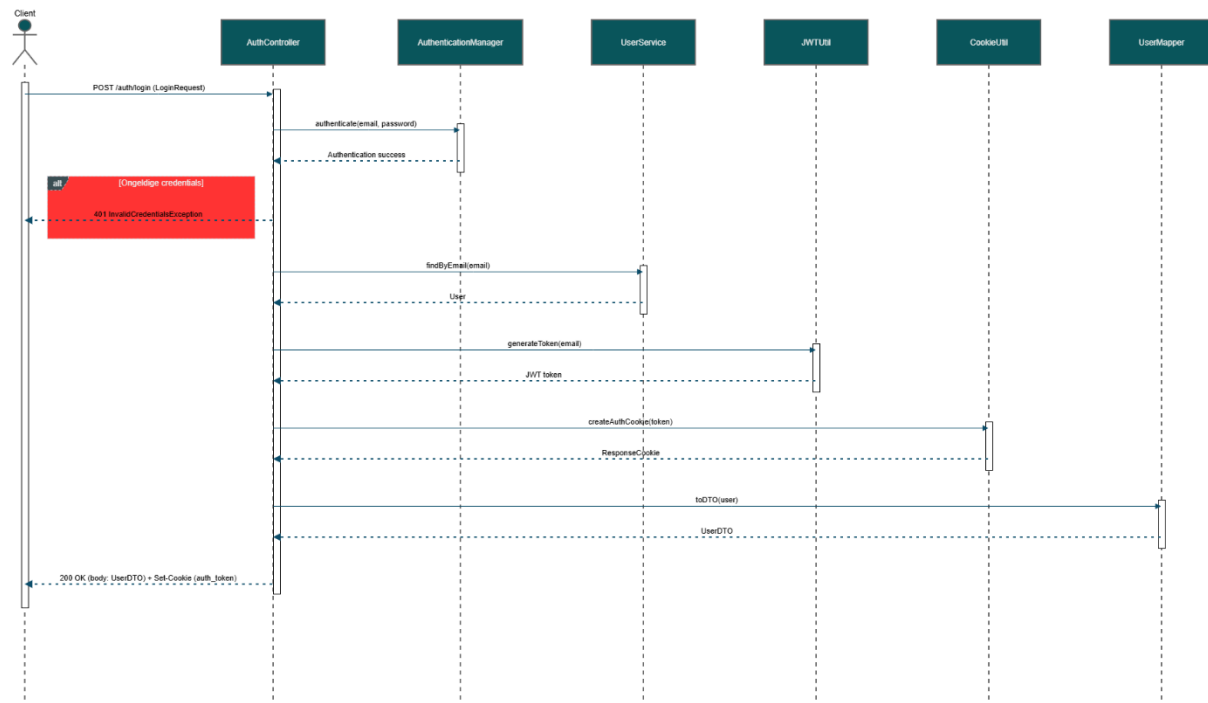
JWT/Security

Het klassendiagram van Security laat de authenticatie-laag zien. De AuthController heeft endpoints voor registratie, login en logout. Bij registratie worden de credentials gevalideerd door de CredentialValidator en wordt het wachtwoord gehasht door de PasswordEncoder. Bij login controleert de AuthenticationManager de gegevens. Na succesvolle authenticatie maakt JWTUtil een token aan en CookieUtil plaatst deze in een HttpOnly cookie. De JWTFilter draait op elk binnenkomend request en controleert of er een geldig token aanwezig is.

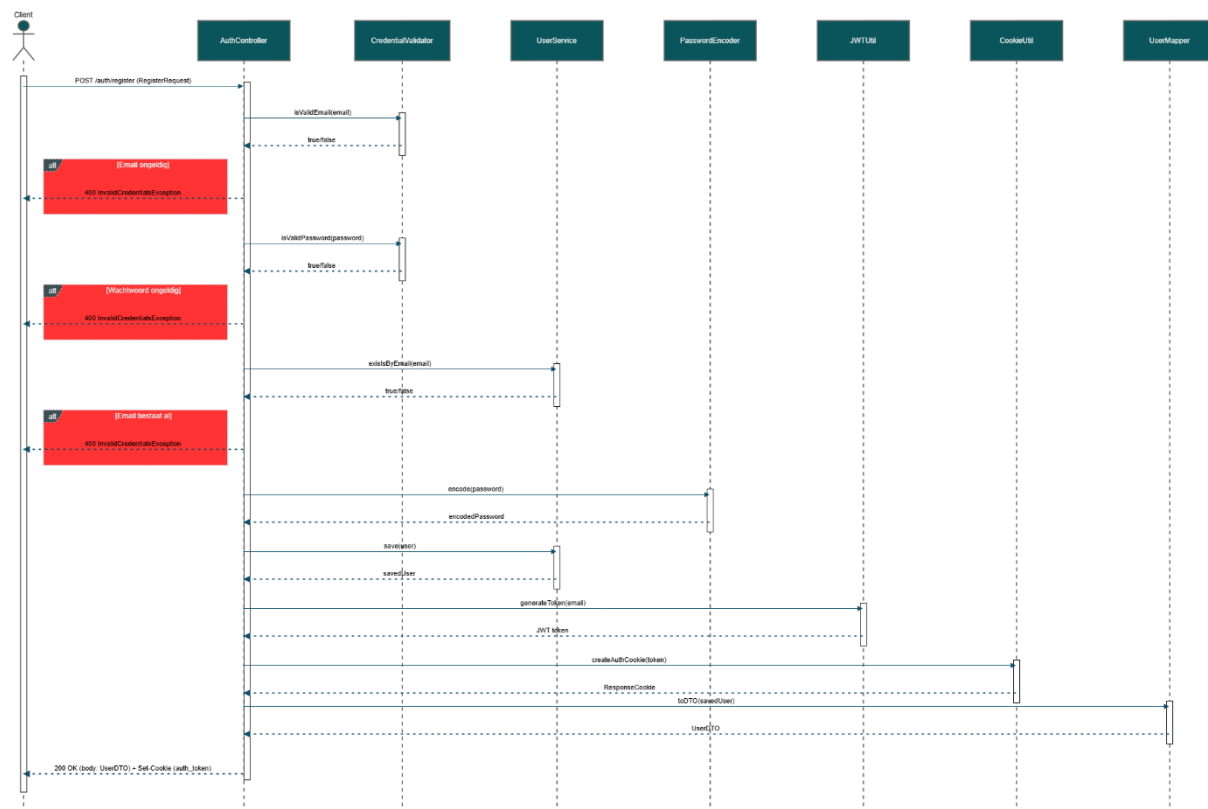


Sequentie diagrammen

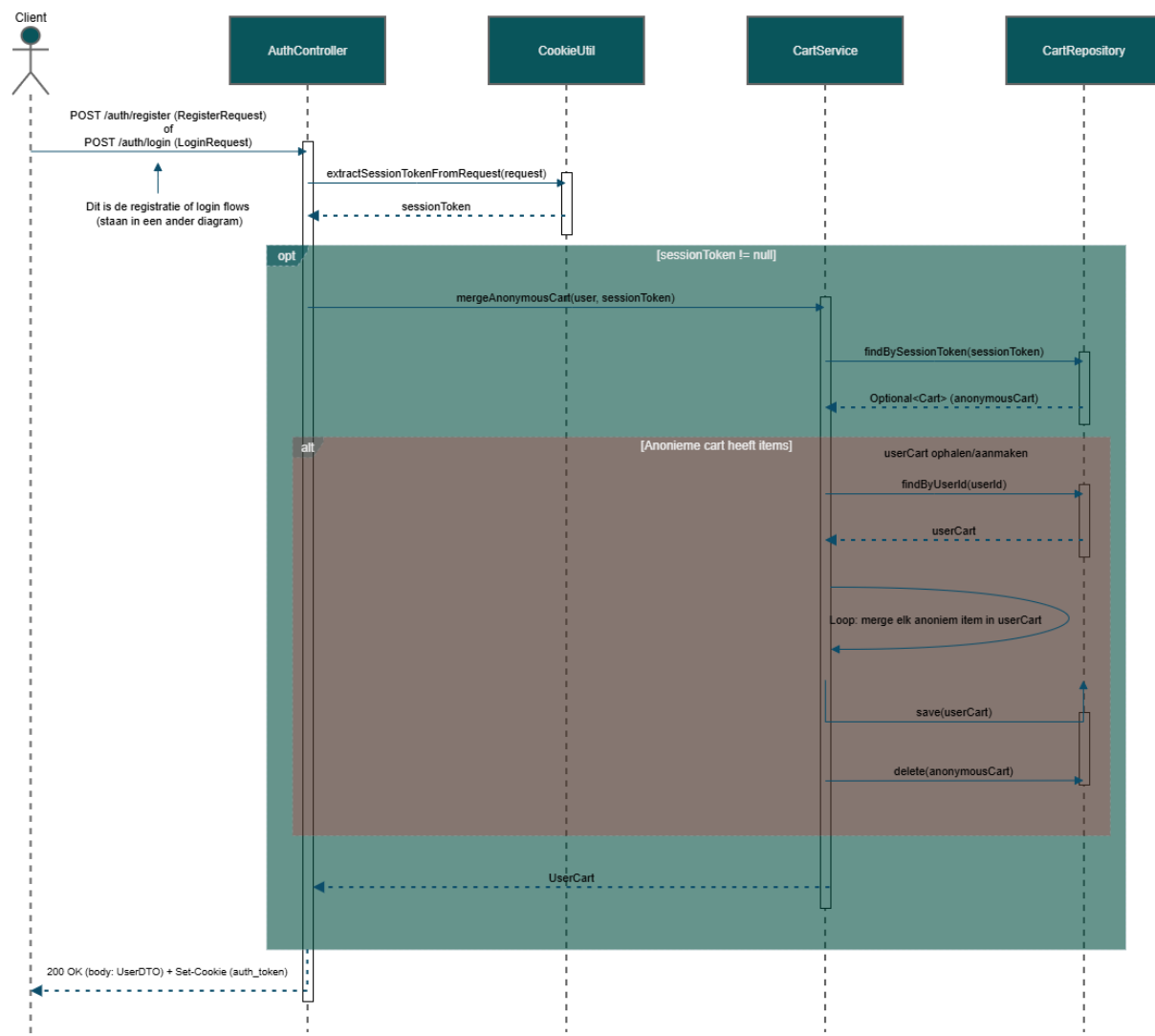
Login flow



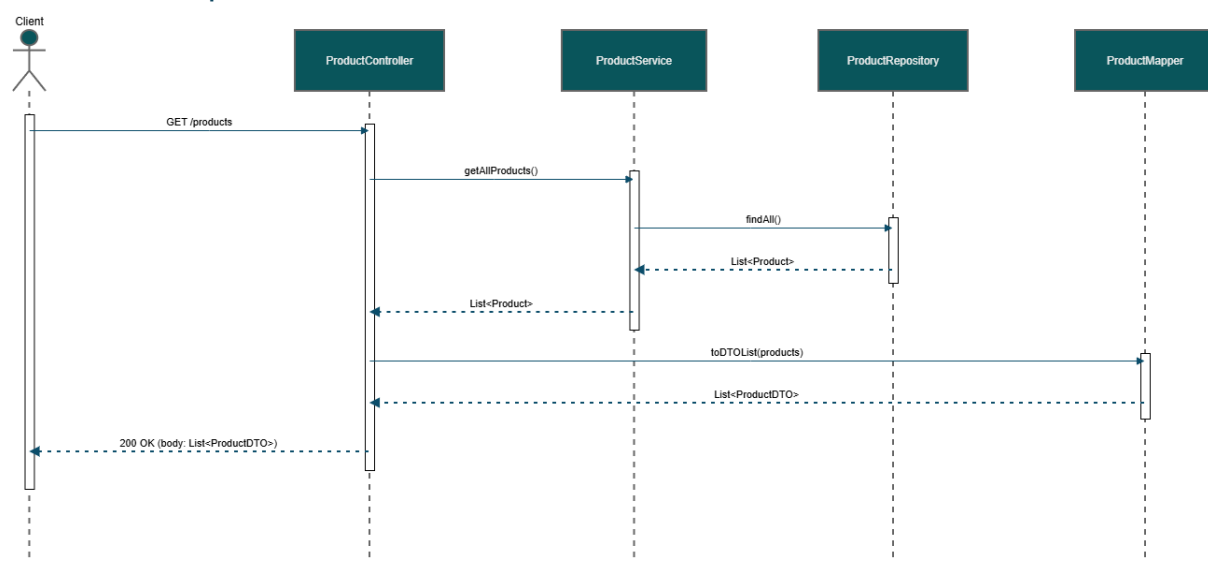
Registratie flow



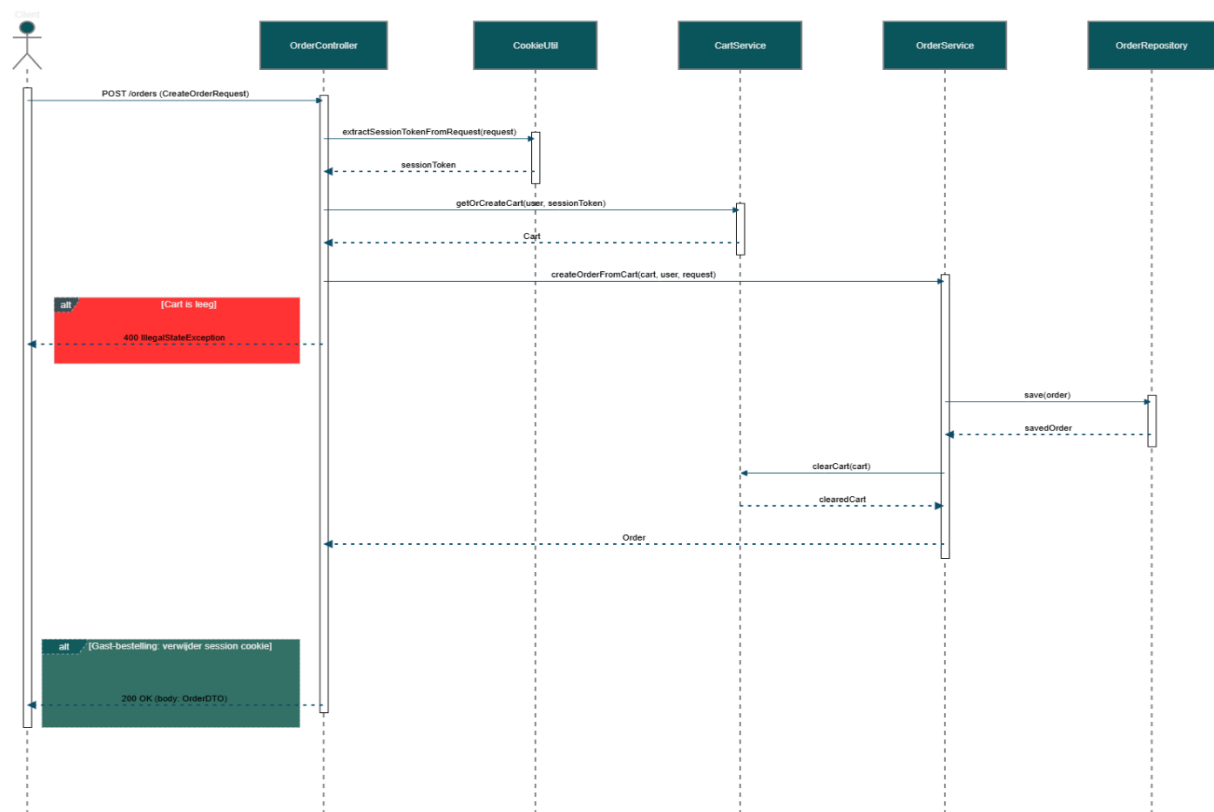
Cart merge flow



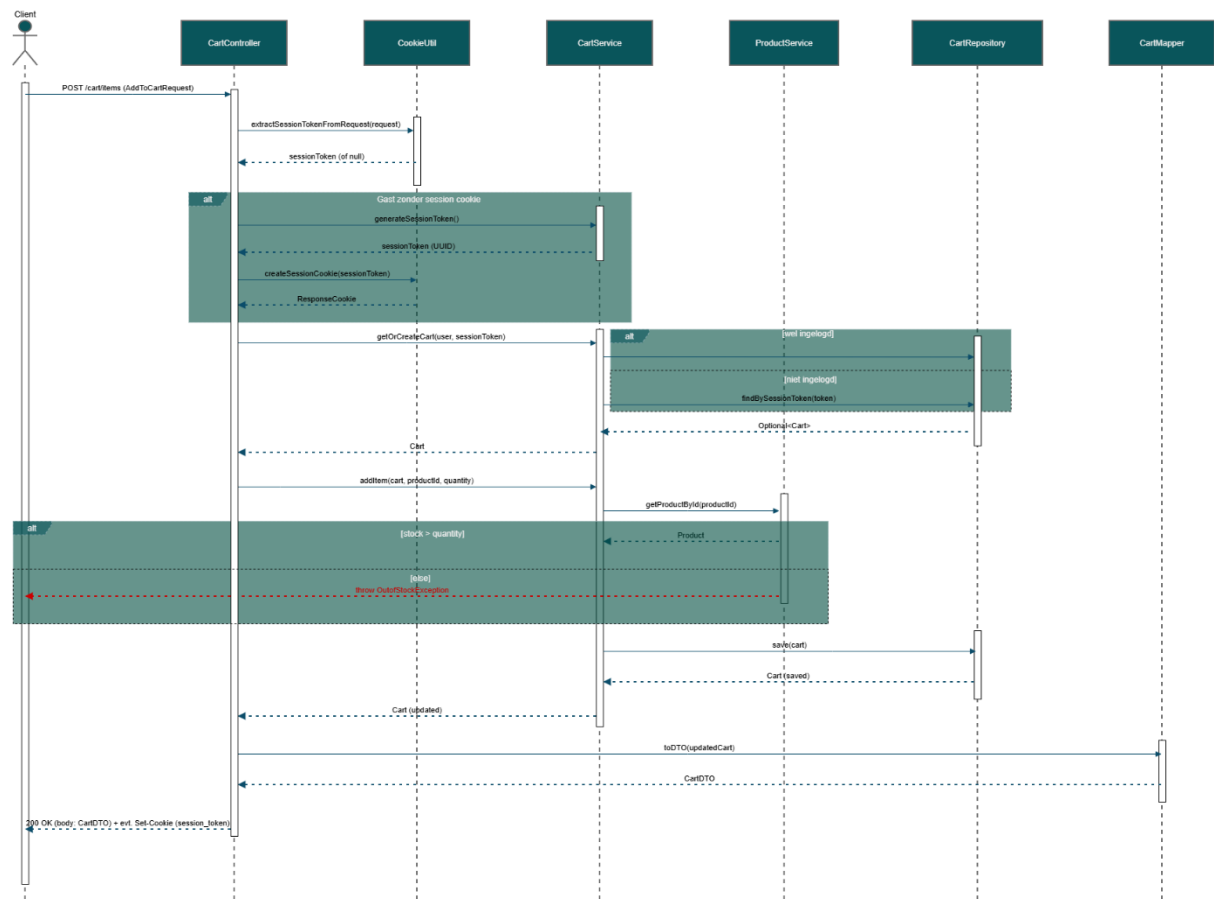
Producten ophalen flow



Order flow



Item toevoegen aan cart flow



End-points en benodigde waardes

Authenticatie

Methode	URL	Auth	Request Body	Response
POST	/auth/register	Nee	email, password, firstName, lastName	200 OK UserDTO + auth cookie
POST	/auth/login	Nee	email, password	200 OK UserDTO + auth cookie
POST	/auth/logout	Nee		200 OK cookie verwijderd

Producten

Methode	URL	Auth	Request Body / Params	Response
GET	/products	Nee		200 OK List<ProductDTO>
GET	/products/{id}	Nee		200 OK ProductDTO
POST	/admin/product	ADMIN	name, price, description, stock, categoryId, brandId, image (multipart)	201 OK ProductDTO
PUT	/admin/product/ {id}	ADMIN	name, price, description, stock, categoryId, brandId, image (multipart)	200 OK ProductDTO
DELETE	/admin/product/ {id}	ADMIN		204 OK No Content
PATCH	/admin/product/ {id}/image	ADMIN	image (multipart)	200 OK ProductDTO

Categorieën

Methode	URL	Auth	Request Body	Response
GET	/categories	Nee		200 OK List<CategoryDTO>
GET	/categories/{id}	Nee		200 OK CategoryDTO
POST	/admin/categories	ADMIN	name, description	201 OK CategoryDTO
PUT	/admin/categories/{id}	ADMIN	name, description	200 OK CategoryDTO
DELETE	/admin/categories/{id}	ADMIN		204 OK No Content

Merken

Methode	URL	Auth	Request Body	Response
GET	/brands	Nee		200 OK List<BrandDTO>
GET	/brands/{id}	Nee		200 OK BrandDTO
POST	/admin/brands	ADMIN	name, description, image (multipart)	201 OK BrandDTO
PUT	/admin/brands/{id}	ADMIN	name, description, image (multipart)	200 OK BrandDTO
DELETE	/admin/brands/{id}	ADMIN		204 OK No Content

Winkelwagen

Methode	URL	Auth	Request Body	Response
GET	/cart	Optioneel		200 OK CartDTO
POST	/cart/items	Optioneel	productId, quantity	200 OK CartDTO
PUT	/cart/items/{itemId}	Optioneel	quantity	200 OK CartDTO
DELETE	/cart/items/{itemId}	Optioneel		200 OK CartDTO
DELETE	/cart	Optioneel		200 OK CartDTO (leeg)

Bestellingen

Methode	URL	Auth	Request Body	Response
GET	/orders	USER		200 OK List<OrderDTO>
GET	/orders/{orderId}	USER (eigenaar)		200 OK OrderDTO
GET	/guest-orders/{orderNumber}	Nee		200 OK OrderDTO
POST	/orders	Optioneel	email, firstName, lastName, shippingAddress	200 OK OrderDTO
GET	/admin/orders	ADMIN		200 OK List<OrderDTO>
GET	/admin/orders/{orderId}	ADMIN		200 OK OrderDTO

Gebruikers

Methode	URL	Auth	Request Body	Response
GET	/users/profile	USER		200 OK UserDTO
PUT	/users/profile	USER	firstName, lastName, currentPassword, newPassword	200 OK UserDTO
GET	/admin/users	ADMIN		200 OK List<UserDTO>
GET	/admin/users/{id}	ADMIN		200 OK UserDTO
POST	/admin/users	ADMIN	email, password, firstName, lastName, role	201 OK UserDTO
PUT	/admin/users/{id}	ADMIN	email, password, firstName, lastName, role	200 OK UserDTO
DELETE	/admin/users/{id}	ADMIN		204 OK No Content

Response DTO's

UserDTO

Veld	Type	Beschrijving
id	Long	Gebruiker-ID
email	String	E-mailadres
firstName	String	Voornaam
lastName	String	Achternaam
role	Role	Gebruikersrol

ProductDTO

Veld	Type	Beschrijving
id	Long	Product-ID
name	String	Productnaam
price	BigDecimal	Prijs
description	String	Beschrijving
imageUrl	String	URL van afbeelding
stock	Integer	Voorraad
categoryId	Long	Categorie-ID
categoryName	String	Categorienaam
brandId	Long	Merk-ID
brandName	String	Merknaam
brandImageUrl	String	URL van merk-afbeelding

CategoryDTO

Veld	Type	Beschrijving
id	Long	Categorie-ID
name	String	Categorienaam
description	String	Beschrijving
productCount	long	Aantal producten in categorie

BrandDTO

Veld	Type	Beschrijving
id	Long	Merk-ID
name	String	Merknaam
description	String	Beschrijving
imageUrl	String	URL van logo/afbeelding
productCount	long	Aantal producten van merk

CartDTO

Veld	Type	Beschrijving
id	Long	Cart-ID
items	List<CartItemDTO>	Lijst van items
totalAmount	BigDecimal	Totaalbedrag
itemCount	int	Aantal items

CartItemDTO

Veld	Type	Beschrijving
id	Long	CartItem-ID
productId	Long	Product-ID
productName	String	Productnaam
productImageUrl	String	URL product-afbeelding
price	BigDecimal	Stuksprijs
quantity	int	Aantal
subtotal	BigDecimal	Subtotaal

OrderDTO

Veld	Type	Beschrijving
id	Long	Bestelling-ID
orderNumber	String	Uniek bestelnummer
status	OrderStatus	Status van bestelling
totalAmount	BigDecimal	Totaalbedrag
shippingAddress	String	Verzendadres
email	String	E-mailadres klant
customerName	String	Naam klant
items	List<OrderItemDTO>	Bestelitems
createdAt	LocalDateTime	Aanmaakdatum

OrderItemDTO

Veld	Type	Beschrijving
id	Long	OrderItem-ID
productId	Long	Product-ID
productName	String	Productnaam
price	BigDecimal	Prijs per stuk
quantity	int	Aantal
subtotal	BigDecimal	Subtotaal

Security

In dit hoofdstuk leg ik uit hoe de beveiliging van de applicatie werkt, zowel aan de frontend- als de backendzijde.

Frontend security OWASP

De frontend is gemaakt in Angular. Angular heeft al standaard beveiliging ingebouwd tegen veel bekende kwetsbaarheden uit de OWASP Top 10. Daardoor is de basis van de frontend al veilig opgezet.

1. Broken Access Control (A01:2021)

Admin onderdelen zijn in de frontend alleen zichtbaar voor gebruikers met de rol ADMIN. Dit gebeurt met een adminGuard op de routes. De echte controle zit wel in de backend. De frontend controle is vooral handig voor de gebruikerservaring en als extra 'kleine' beveiligingslaag.

2. Cryptographic Failures (A02:2021)

Wachtwoorden worden niet opgeslagen in de frontend. Ze worden alleen gebruikt bij registreren en inloggen. Daarna ontvangt de frontend alleen gebruikersgegevens zonder wachtwoord. Cookies worden alleen via HTTPS verstuurd.

3. Injection (A03:2021)

Angular beschermt standaard tegen XSS. Waarden in templates worden automatisch veilig weergegeven.

4. Insecure Design (A04:2021)

De frontend is overzichtelijk opgebouwd. Services communiceren met de backend, guards beschermen routes en interceptors regelen authenticatie. Gevoelige onderdelen, zoals adminfuncties, zijn apart afgeschermd.

5. Security Misconfiguration (A05:2021)

Alle API-calls lopen via een centrale service. Daarbij worden cookies steeds correct meegestuurd. Ook is de backend zo ingesteld dat alleen het eigen frontend domein toegang heeft (CORS).

6. Vulnerable and Outdated Components (A06:2021)

De applicatie gebruikt actuele Angular packages. Het blijft wel belangrijk om dependencies regelmatig te updaten. Dit kan gecontroleerd worden met npm audit.

7. Identification and Authentication Failures (A07:2021)

De login werkt met HttpOnly cookies. De JWT wordt niet opgeslagen in localStorage of sessionStorage. Daardoor kan JavaScript er niet bij. Bij een 401 fout wordt de gebruiker uitgelogd.

8. Software and Data Integrity Failures (A08:2021)

De frontend wordt gebouwd als een statische Angular build. Er wordt geen code geladen uit onbekende bronnen. Ook worden geen losse externe scripts gebruikt.

9. Security Logging and Monitoring Failures (A09:2021)

Bij een 401 fout reageert de frontend direct door de gebruiker uit te loggen. API-fouten worden opgevangen en waar nodig getoond. Logging van mislukte loginpogingen en ongeldige tokens gebeurt vooral in de backend.

10. SSRF (A10:2021)

SSRF is niet echt van toepassing op de frontend. De frontend stuurt alleen requests naar de eigen backend. Er worden geen externe servers aangeroepen.

Backend security

Authenticatie

Bij het inloggen of registreren wordt een JWT-token aangemaakt door JWTUtil. De token is ondertekend met HMAC256 en een geheime sleutel uit application.properties. In de token zit het e-mailadres van de gebruiker en een vervaltijd van 6 uur.

De token wordt niet als JSON teruggegeven, maar opgeslagen als HttpOnly cookie genaamd auth_token. Zo kan JavaScript er niet bij. De cookie heeft ook de Secure- en SameSite-flags voor extra bescherming.

JWTFilter

Bij elk binnenkomend verzoek voert de JWTFilter de volgende stappen uit:

- Het auth_token cookie wordt opgehaald.
- De token wordt gevalideerd.
- De bijbehorende gebruiker wordt opgehaald en in de SecurityContext geplaatst.
- Is de token ongeldig of verlopen, dan wordt het verzoek als anoniem behandeld.

Autorisatie

In SecurityConfig staat precies wie welke routes mag benaderen:

- **Publiek:** routes zoals /products/**, /auth/** en /cart/**
- **Ingelogd:** zoals /orders en /users/profile
- **Alleen admin:** alles onder /admin/**

Bovenop de URL-beveiliging staat ook @PreAuthorize("hasRole('ADMIN')") op alle admin-controllers. Dit is een extra beveiligingslaag.

Wachtwoordbeveiliging

Wachtwoorden worden nooit als plaintext opgeslagen. Bij registratie wordt het wachtwoord gehasht met BCrypt. Bij het inloggen vergelijkt Spring Security het ingevoerde wachtwoord met de opgeslagen hash.

Inputvalidatie

Invoer wordt op meerdere plekken gecontroleerd:

- CredentialValidator controleert of het e-mailadres geldig is en of het wachtwoord voldoet aan de eisen (8–30 tekens, minimaal 1 cijfer, 1 kleine letter, 1 hoofdletter en 1 speciaal teken).
- Op request-DTO's staan annotaties zoals @NotBlank, @Email en @Valid, die Spring automatisch valideert.
- Bij registratie wordt ook gecontroleerd of het e-mailadres al in gebruik is.

CORS

Alleen het frontend-domein mag verzoeken doen naar de backend. Toegestane headers zijn beperkt tot Authorization, Content-Type en X-XSRF-TOKEN.

CSRF

Via CookieCsrfTokenRepository in de securityconfig krijgt de frontend een CSRF-token dat Angular automatisch meestuurt als header. Dit is actief op alle niet-publieke endpoints.

Stateless sessies

De applicatie houdt geen server-side sessies bij (SessionCreationPolicy.STATELESS). Alle authenticatie verloopt via de JWT in de cookie. Dit maakt de applicatie schaalbaarder en voorkomt sessiegerelateerde aanvallen.

Gastwinkelwagen

Niet-ingelogde gebruikers krijgen een session_token cookie (een willekeurige UUID) waarmee hun winkelwagen wordt bijgehouden. Bij het inloggen wordt deze winkelwagen samengevoegd met de gebruikerswinkelwagen en wordt het session_token verwijderd.